Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

ĐỒ ÁN THIẾT KẾ LUẬN LÝ UART TO USB BRIDGE

Instructor: Phạm Kiều Nhật Anh

Name	Student ID
Lục Tấn Phúc	2352935

Ho Chi Minh City, December 2025

1 PicoRV32 SoC UART Testbench: Single Core

1.1 Overview

In this part of the lab, we verify a simple RISC-V SoC composed of the `picorv32` CPU core, a 64 KiB block RAM (`bram_64k`), and a memory-mapped UART (`uart_simple`). The system is wrapped by the top-level module `top` (file `top_btn_reset_clean.v`), which connects external pins:

- `clk`: 125 MHz input clock.
- `rst_btn`: active-high reset button.
- `uart_rx`: UART receive line from PC.
- `uart_tx`: UART transmit line to PC.

A firmware image stored in `firmware.mem` is loaded into `bram_64k` at reset. This firmware runs on the PicoRV32 core and sends the text string `"Hi!\r\n"` over the UART. To validate the entire path from CPU to serial line, we build a dedicated Verilog testbench that:

- Generates the 125 MHz clock and applies a reset pulse.
- Lets the CPU execute the firmware stored in BRAM.
- Passively listens on `uart_tx`, decodes UART frames into bytes, and prints them to the simulator console.
- Produces a waveform (VCD) to inspect bit-level UART timing.

1.2 UART SoC Testbench

1.2.1 Testbench Code

The complete testbench is shown in Listing 1. It instantiates the top-level SoC, provides stimulus for the clock and reset, and contains a task `uart_capture_byte` which recovers bytes from the serial line.

Listing 1: PicoRV32 SoC UART testbench

```
`timescale 1ns/1ps

// Testbench for top_btn_reset_clean (module top)
module tb_top;

    // Match DUT parameters
    localparam integer CLK_HZ = 125_000_000;
    localparam integer BAUD = 115200;

    // UART TX clock divider = clocks per bit
    localparam integer TX_DIV = (CLK_HZ + BAUD/2) / BAUD;

    reg clk;
    reg rst_btn;
    reg uart_rx;
```

```
wire uart_tx;

// Device Under Test (DUT): PicoRV32 + BRAM + UART
top #(
    .CLK_HZ(CLK_HZ),
    .BAUD (BAUD)
) dut (
    .clk (clk),
    .rst_btn (rst_btn),
    .uart_rx (uart_rx),
    .uart_tx (uart_tx)
);

// 125 MHz clock: 8 ns period
initial clk = 1'b0;
always #4 clk = ~clk;

// Task: capture one UART byte (8N1, LSB first) from uart_tx
task automatic uart_capture_byte(output reg [7:0] data);
    integer i;
    begin
        // Wait for start bit (falling edge)
        @(negedge uart_tx);

        // Move to the middle of first data bit
        repeat (TX_DIV/2) @(posedge clk);

        // Sample 8 data bits, LSB first
        for (i = 0; i < 8; i = i + 1) begin
            repeat (TX_DIV) @(posedge clk);
            data[i] = uart_tx;
        end

        // Wait through stop bit
        repeat (TX_DIV) @(posedge clk);
    end
endtask

integer k;
reg [7:0] ch;

initial begin
    // Dump waveform
    $dumpfile("tb_top.vcd");
    $dumpvars(0, tb_top);

    // Initial values
    rst_btn = 1'b1; // reset button pressed (active-high)
    uart_rx = 1'b1; // RX line idle (no incoming data)

    // Hold reset for some cycles
    repeat (50) @(posedge clk);
    rst_btn = 1'b0; // release reset
```

```
        $display("%t␣:␣Reset␣released", $time);

        // Let CPU boot and start executing firmware
        repeat (2000) @(posedge clk);

        // Capture 5 transmitted bytes
        for (k = 0; k < 5; k = k + 1) begin
            uart_capture_byte(ch);
            if (ch >= 8'd32 && ch < 8'd127)
                $display("%t␣:␣UART␣TX␣byte␣=␣0x%02x␣(%c)", $time, ch, ch);
            else
                $display("%t␣:␣UART␣TX␣byte␣=␣0x%02x␣(.)", $time, ch);
        end

        // Extra time for safety, then finish
        #1_000_000;
        $finish;
    end

endmodule
```

1.2.2 Testbench Flow

The behaviour of the testbench can be summarized as follows:

1. **Clock generation.** The `always #4 clk = ~clk;` block produces a 125 MHz clock (8 ns period) that drives the DUT, matching the board clock.
2. **Reset sequence.** The initial block sets `rst_btn = 1` to assert reset. After 50 clock cycles, the reset button is released (`rst_btn = 0`), and the internal synchronized reset inside `top` lets the CPU start running.
3. **CPU boot and firmware execution.** The BRAM module `bram_64k` loads the program from `firmware.mem` using `$readmemh`. The PicoRV32 core begins fetching instructions via its `mem_*` bus interface and executes the firmware. Part of this program writes the bytes of the string `"Hi!\r\n"` to the memory-mapped UART transmit register at address `0x1000_0000`.
4. **UART transmission.** The top-level address decoder in `top_btn_reset_clean.v` converts writes to the UART region into `tx_start` and `tx_data` for the `uart_simple` block. The UART module uses the parameters `CLK_HZ` and `BAUD` to compute `TX_DIV` (clocks per bit) and shifts out each frame on `uart_tx` with 8 data bits, no parity, 1 stop bit (8N1).
5. **Byte capture in the testbench.** The task `uart_capture_byte` in the testbench waits for a falling edge on `uart_tx` (start bit), then samples the line every `TX_DIV` clock cycles to reconstruct the transmitted byte. After receiving 8 data bits and skipping the stop bit, the recovered byte is printed on the console.

1.2.3 Function of the Testbench

Overall, the testbench verifies the following aspects of the design:

- **Reset and boot.** Confirms that the SoC correctly releases from reset and that the CPU starts executing instructions from BRAM.

- **Memory-mapped UART interface.** Validates that the firmware writes to the correct UART address, that the top-level address decoder generates the proper `tx_start` and `tx_data` signals, and that `uart_simple` responds by sending bytes.
- **UART timing at 115200 baud.** Checks that the baud-rate divider `TX_DIV` is correct for a 125 MHz clock and that each UART bit appears with the expected duration on `uart_tx`.
- **End-to-end data correctness.** Demonstrates that the bytes coming out of the UART line match exactly the string embedded in the firmware, proving the CPU–BRAM–bus–UART path is functionally correct.

1.3 Simulation Output and Interpretation

1.3.1 TCL Console Output

When running the behavioural simulation in Vivado, the simulator TCL console prints messages from the `$display` statements in the testbench:

```
396000 : Reset released
607292000 : UART TX byte = 0x48 (H)
693964000 : UART TX byte = 0x69 (i)
780764000 : UART TX byte = 0x21 (!)
867564000 : UART TX byte = 0x0d (.)
954364000 : UART TX byte = 0x0a (.)
```

Here, the timestamps are in picoseconds. The line

```
396000 : Reset released
```

corresponds to the moment when `rst_btn` is driven low after 50 clock cycles. The subsequent lines show the bytes that the testbench recovers from `uart_tx`:

- `0x48` = 72 (decimal) = character `'H'`.
- `0x69` = `'i'`.
- `0x21` = `'!'`.
- `0x0d` = carriage return `\r`.
- `0x0a` = line feed `\n`.

Thus, the console confirms that the firmware sends the string `"Hi!\r\n"` over the UART and that the testbench correctly reconstructs the data.

The intervals between consecutive bytes are approximately $86.8\,\mu s$. This matches the expected UART frame time at 115200 baud:

$$\text{bit time} = \frac{1}{115200} \approx 8.68\,\mu s, \quad \text{frame time} \approx 10 \times 8.68\,\mu s = 86.8\,\mu s,$$

since each 8N1 frame contains 1 start bit, 8 data bits, and 1 stop bit.

1.3.2 Waveform in the 0.6 ms–1.0 ms Range

The Vivado waveform viewer (XSIM) provides a detailed view of the signals during simulation. When zooming in to the 0.6 ms–1.0 ms interval, we see the individual UART frames corresponding to the bytes shown above.

1.3.2.1 UART line behaviour.

In this time window:

- `uart_tx` is normally high (1) when idle.
- For each transmitted byte, `uart_tx` drops low for one bit time (start bit), then toggles for 8 bit times according to the data bits, and finally returns high for at least one bit time (stop bit).
- The bit period on `uart_tx` measures approximately 8.68 μs, which matches

$$\text{TX_DIV} \times T_{\text{clk}} = 1085 \times 8\,\text{ns} \approx 8.68\,\mu\text{s},$$

  where `TX_DIV` is computed in the UART module and exposed as a parameter in the testbench.

1.3.2.2 Register `ch[7:0]` and loop index `k`.

The testbench internal signals also appear on the waveform:

- `ch[7:0]` holds the last byte captured by `uart_capture_byte`. For example, around $t \approx 0.607$ ms, `ch` updates to `0x48`, indicating that the first frame (character `'H'`) has just been fully received.
- `k[3:0]` is the loop counter in the `for` loop. It increments each time a new byte is captured. When `k = 1`, the first byte has been received; when `k = 5`, the full string `"Hi!\r\n"` has been processed.

1.3.2.3 Connection to the underlying code.

The waveform observed in this 0.6 ms–1.0 ms window is a direct consequence of the following design elements:

- The **firmware** in `firmware.mem` executes on the `picorv32` core and writes the sequence of bytes `0x48, 0x69, 0x21, 0x0d, 0x0a` to the UART transmit register.
- The **address decoder** in `top_btn_reset_clean.v` interprets these memory writes and asserts `tx_start` and `tx_data` for the `uart_simple` module.
- The **UART transmitter** in `uart_simple.v` shifts out each byte on `uart_tx` using the baud-rate divider `TX_DIV`, generating the observed 8N1 frames.
- The **testbench task** `uart_capture_byte` samples `uart_tx` in the middle of each bit period (also using `TX_DIV`) and updates the register `ch[7:0]`, which is visible on the waveform and printed to the console.

1.4 Waveform Demonstration

This subsection presents the UART transmission waveform of the PicoRV32 SoC during the interval from 0.6 ms to 1.0 ms, where the firmware sends the string `"Hi!\r\n"` over the memory-mapped UART. Five captured waveform snapshots are shown below, corresponding to the moments from left to right: 1.0 ms, 0.9 ms, 0.8 ms, 0.7 ms, and 0.6 ms.

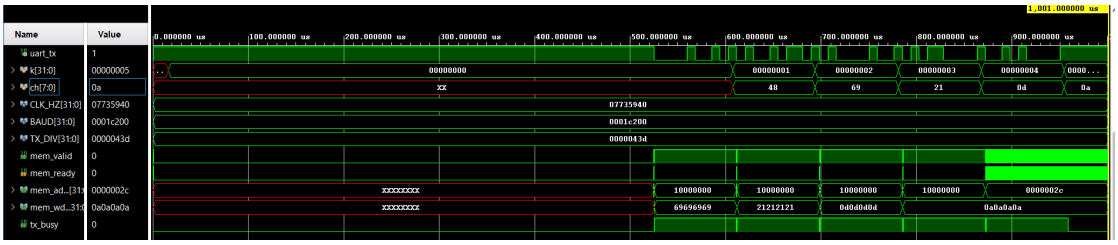
1.4.1 Overview of Signals in the Waveforms

Each figure contains the following important signals:

- `uart_tx`: the actual serial output line.
- `ch[7:0]`: the byte decoded by the testbench after each UART frame completes.
- `k[31:0]`: loop counter in the testbench indicating how many characters have been captured.
- `mem_valid`, `mem_ready`: handshake signals from the PicoRV32 memory bus.
- `mem_addr`, `mem_wdata`: show when the CPU writes to the UART MMIO address.
- `tx_busy`: asserted while `uart_simple` transmits bits of a UART frame.

These signals allow us to correlate CPU activity, bus transactions, UART shifting, and testbench decoding.

1.4.2 Waveform at 1.0 ms



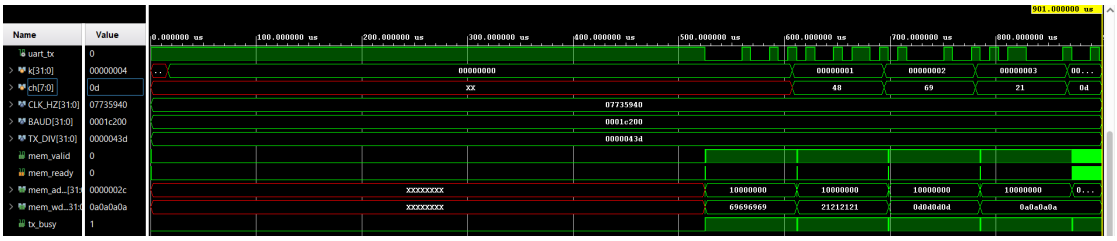
Hình 1: UART waveform at 1.0 ms. The fifth byte (`0x0A`, newline) has been decoded.

At 1.0 ms, the testbench shows:

- `k = 5`: five characters have been received.
- `ch = 0x0A`: the last byte decoded is line feed (`\n`).
- `uart_tx` has returned to idle (logic 1), meaning the UART transmitter has completed all frames.

The memory-bus signals (`mem_addr = 0x2C`, `mem_wdata = 0x0A0A0A0A`) reflect the CPU's internal data movement after printing the final character.

1.4.3 Waveform at 0.9 ms



Hình 2: UART waveform at 0.9 ms, showing completion of the carriage-return byte `0x0D`.

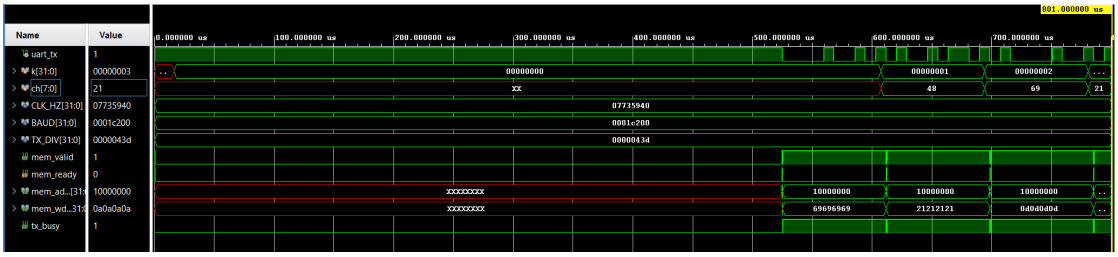
At this moment:

- `ch = 0x0D`: the testbench has just sampled the carriage return character.
- `k = 4`: four bytes have been fully received.
- `tx_busy = 1`: the UART transmitter is currently outputting the next byte (`0x0A`).

  This timestamp corresponds exactly with the console output:

$$t = 867564000\,\text{ps} \rightarrow \texttt{0x0D}.$$

1.4.4 Waveform at 0.8 ms



Hình 3: UART waveform at 0.8 ms, showing decoding of exclamation mark (`0x21`).

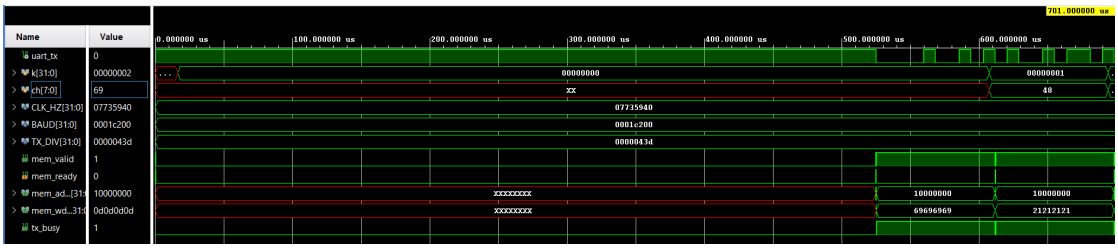
Here:

- `ch = 0x21`: ASCII '`!`'.
- `k = 3`: indicates the third character in the sequence.
- `tx_busy` is asserted while the UART outputs the CR byte next.

  The data bits visible on `uart_tx` match the binary of `0x21`:

$$0010\,0001_{(2)}.$$

1.4.5 Waveform at 0.7 ms



Hình 4: UART waveform at 0.7 ms, showing decoding of `'i'` (`0x69`).

At this time:

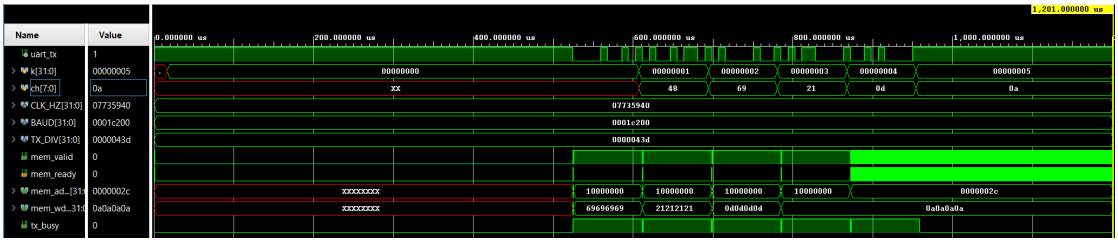
- `ch = 0x69`: ASCII `'i'`.
- `k = 2`: the second byte received.
- `uart_tx` data bits correspond to `0x69 = 01101001`.

You may also notice a write on the memory bus where:

$$\texttt{mem_addr = 0x2C}, \quad \texttt{mem_wdata = 0x0D0D0D0D},$$

coming from the CPU as part of the firmware loop.

1.4.6 Waveform at 0.6 ms



Hình 5: UART waveform at 0.6 ms, showing decoding of `'H'` (`0x48`).

This is where the first UART frame finishes:

- `ch = 0x48`: the testbench successfully reconstructs the byte.
- `k = 1`: indicates the first decoded character.

The bit pattern on `uart_tx` matches:

$$0_\text{start},\ 00010010_{(2)}_\text{data(LSB}\rightarrow\text{MSB)},\ 1_\text{stop}.$$

The time difference between this frame and the next ($\approx 86.8\,\mu$s) matches the UART configuration:

$$\text{bit time} = 1085 \times 8\,\text{ns}, \quad 10 \times \text{bit time} \approx 86.8\,\mu\text{s}.$$

1.4.7 Relationship to RTL and Firmware

The waveform fragments are a direct consequence of:

- The firmware writing bytes `0x48, 0x69, 0x21, 0x0D, 0x0A` to the UART MMIO address at `0x1000_0000`.
- The address decoder in `top_btn_reset_clean.v` generating `tx_start` and `tx_data`.
- The UART transmitter in `uart_simple.v` shifting out bits based on `TX_DIV`.
- The testbench task `uart_capture_byte` sampling `uart_tx` precisely at each bit center.

Thus, what appears on the waveform is a complete, cycle-accurate representation of the CPU–bus–UART pipeline working together.

Together, the TCL console output and the waveform confirm that the SoC boots correctly, that the firmware executes as intended, and that the UART path from CPU to serial line operates at the correct baud rate and transmits the correct data bytes.